Sched It

Implementa un proyecto web Django que permita gestionar citas (no románticas – aunque también valdría –).

Por ejemplo, quiero pedir una cita para la peluquería el 22 de noviembre de 8:00h a 9:00h. Al ser un sistema con acceso de múltiples usuarios, habrá citas que estén ocupadas ya.

En este ejercicio no se considera el *concepto* (motivo) de la cita. Son "citas genéricas".

1. Requerimientos

Para que puedas trabajar con normalidad en este ejercicio debes tener instalado en tu máquina:

1. `uv` → Gestor de paquetería y proyectos Python.
2. `just` → Lanzador de comandos como recetas.

2. Puesta en marcha

Se proporciona una *receta* `just` para la puesta en marcha del proyecto:

```
just setup
```

¿Qué ha ocurrido?

- Se ha creado un entorno virtual en la carpeta `.venv`
- Se han instalado las dependencias del proyecto.
- Se ha creado un proyecto Django en la carpeta `main`
- Se han aplicado las migraciones iniciales del proyecto.
- Se ha creado un *superusuario* con credenciales: `admin` - `admin`
- Se ha establecido el *timezone* a `Atlantic/Canary` en `settings.py`

3. Aplicaciones

Habrá que añadir, al menos, las siguientes aplicaciones:

`accounts`	Gestión de autenticación.
`appointments`	Gestión de citas.
`slots`	Gestión de franjas horarias.

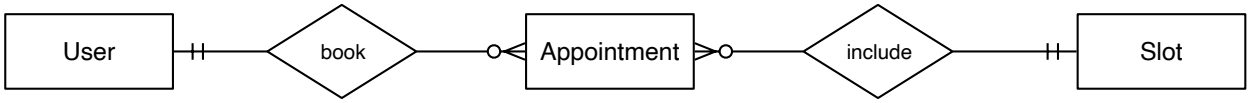
CC BY 4.0

Se proporciona una *receta* `just` para añadir una aplicación:

```
just startapp <app>
```

Esta receta no sólo crea la carpeta de la aplicación sino que añade la línea correspondiente de configuración en la variable `INSTALLED_APPS` del fichero `settings.py`.

4. Modelos



Un usuario (`User`) reserva citas (`Appointment`) y una cita incluye franjas horarias (`Slot`).

4.1. `slots.Slot`

Campo	Tipo
`start_at`(*)	*time*
`end_at`(*)	*time*

Notas:

- No son datetimes, sólo almacenan horas.
- La *hora de comienzo* y la *hora de finalización* **juntos** deben ser valores únicos.
- La ordenación por defecto debe ser por `start_at` y `end_at`, ambos de forma *ascendente*.
- Implementa la representación del objeto en formato: `hh:mm - hh:mm`
- Ayúdate de la función `strftime` para mostrar las horas y los minutos.

4.2. `appointments.Appointment`

Campo	Tipo
`date`(*)	*date*
`slot`(*)	*foreign key*
`user`(*)	*foreign key*

Notas:

- El campo `date` no es un datetime, sólo almacena una fecha.
- Debes tener en cuenta las claves ajenas necesarias (*si procede*). No te olvides de `related_name`.
- La fecha y el "slot" **juntos** deben ser valores únicos.

 CC BY 4.0

- La ordenación por defecto debe ser por fecha y slot, ambos de forma *ascendente*.
- Implementa la representación del objeto en formato: `yyyy-mm-dd (hh:mm - hh:mm)`
- Implementa su URL canónica.

4.3. Carga de datos

Una vez que hayas **creado los modelos** asegúrate de que tu proyecto pasa los *tests core*. Se proporciona una *receta* `just` para ello:

```
just test tests/test_core.py
```

Ahora **aplica las migraciones** y, si todo ha ido bien, procede a la carga de datos iniciales. Se proporciona una *receta* `just` para ello:

```
just load-data
```

5. Requisitos de diseño

El proyecto deberá cumplir con los siguientes **requisitos de diseño** a nivel de organización interna del código:

- Utiliza formularios de modelo para todos los casos propuestos.
- Utiliza URL canónica para los modelos que procedan.

6. URLs

6.1. `main.urls`

`/`

- Debe redirigir a `/appointments/`

 CC BY 4.0

6.2. `accounts.urls`

`/login/` ⇒ `accounts.views.user_login()`

- Inicio de sesión.
- Se debe solicitar nombre de usuario y contraseña.
- Si el usuario y/o contraseña son incorrectos, se debe mostrar un mensaje de error: Incorrect username or password.
- Si el usuario y contraseña son correctos, se debe iniciar la correspondiente sesión y redirigir al índice del proyecto.
- Hay que tener en cuenta la redirección en el caso de que tengamos un parámetro `next` desde GET.
- Debe existir un enlace para registrarse ("signup").

`/logout/` ⇒ `accounts.views.user_logout()`

- Cierre de sesión.
- Una vez cerrada la sesión, se debe redirigir al índice del proyecto.

`/signup/` ⇒ `accounts.views.user_signup()`

- Registro de usuario.
- Campos a desplegar en el formulario:
  - Nombre de usuario.
  - Contraseña.
  - Nombre.
  - Apellidos.
  - Correo electrónico.
- Aquí se debe crear también el perfil del usuario registrado. Será un perfil "vacío" pero vinculado con el usuario.
- Si el registro fue exitoso, se debe redirigir al índice del proyecto.
- Debe existir un enlace para logearse ("login").

CC BY 4.0

6.3. `appointments.urls`

`/appointments/` ⇒ `appointments.views.appointment_list()`

- Listado con las citas del usuario logeado.
- El formato de cada cita debe ser: `yyyy-mm-dd (hh:mm - hh:mm)`
- Debe existir un enlace al detalle de cada cita.
- La ordenación de las citas vendrá *por defecto* a partir del modelo `Appointment`.
- Debe existir un enlace para añadir una nueva cita.
- Si no hubiera citas se debe mostrar el mensaje: No appointments found.
- Sólo debe ser accesible si el usuario está logeado.

`/appointments/34/` ⇒ `appointments.views.appointment_detail()`

- Detalle de la cita con `pk=34`
- Debe aparecer la fecha con formato: `Date: yyyy-mm-dd`
- Debe aparecer el "slot" con formato: `Slot: hh:mm - hh:mm`
- Debe aparecer un enlace para volver al listado de citas.
- Si un usuario trata de acceder a una cita que no le pertenece, hay que devolver un 403.
- Sólo debe ser accesible si el usuario está logeado.

`/appointments/add/` ⇒ `appointments.views.add_appointment()`

- Añadir una nueva cita.
- Desplegar un **formulario de modelo** con fecha y "slot".
- Si la cita se añadió con éxito redireccionar al detalle de la cita que se acaba de añadir.
- Debe aparecer un enlace para cancelar y volver al listado de citas.
- Sólo debe ser accesible si el usuario está logeado.

7. Administración

Los siguientes modelos deben estar accesibles desde la **interfaz administrativa** de Django:

- `appointments.Appointment`
- `slots.Slot`

 CC BY 4.0